Lab Report – EE-170L Computer Programming (Lab 14)

NAME	NAEEM UL HAQ
Reg.no	0665
SEMESTER	2nd
LAB	14 cpp
DATE	18/5/2026

❖ Objectives

· To design a modular C++ program that calculates student GPA and grades relative to the class topper.

· To implement separate modules for input, weighted marks calculation, average calculation, GPA scaling, grade assignment, and report generation.

· To demonstrate the use of arrays, loops, functions, and conditional statements in a structured program.

· To produce a final transcript for each student showing marks, GPA, and grades.

❖ Theory

1. Input Module

· Purpose: Collect student names, subjects, marks, and credit hours.

· Uses nested loops: outer loop for students, inner loop for subjects.

· Stores weighted marks and credits in arrays.

2.Weighted Marks Module

- Formula:

Weighted Marks=(0.25× Sessional)+(0.25× Mid)+(0.50× Final)

- Ensures fair distribution of marks across assessments.

3.Average Calculation Module

- omputes weighted average marks for each student:

Average=∑ (Marks× Credits) /∑Credits

- Accounts for subject credit hours.

4.Relative GPA Module

- Scales GPA relative to topper’ s average:

Relative GPA=Student AvgTopper / topper average x 4.0

- Ensures topper always gets GPA = 4.0.

5. Grade Assignment Module

- Normalizes marks against topper’ s marks:

Normalized=Student Marks / Topper Marks× 100

- Assigns letter grades using an if-else ladder (A, B+, C, D, F).

6. **Report Module**

- Prints transcript for each student: name, GPA, subject marks, and grades.
- Combines results from all modules into a clear output.

❖ Program (full code)

```
#include<iostream>
#include<string>
using namespace std;


// Weighted Marks Module
double calculateMarks(double sessional, double mid, double final) {
 return (sessional * 0.25) + (mid * 0.25) + (final * 0.50);
}


// Average Calculation Module
double calculateAverage(double subjectMarks[], double subjectCredits[], int
subjectCount) {

  double total = 0, credits = 0;
  for(int i=0; i<subjectCount; i++) {
    total += subjectMarks[i] * subjectCredits[i];
    credits += subjectCredits[i];
  }
  return total / credits;
```

```
}



// Relative GPA Module
double calculateRelativeGPA(double studentAvg, double topperAvg) {
  double gpa = (studentAvg / topperAvg) * 4.0;
  return (gpa > 4.0) ? 4.0 : gpa;
}

// Grade Assignment Module
string assignGrade(double studentMarks, double topperMarks) {
  double normalized = (studentMarks / topperMarks) * 100;
  if(normalized >= 85) return "A";
  else if(normalized >= 80) return "A-";
  else if(normalized >= 75) return "B+";
  else if(normalized >= 70) return "B";
  else if(normalized >= 65) return "B-";
  else if(normalized >= 60) return "C+";
  else if(normalized >= 55) return "C";
  else if(normalized >= 50) return "C-";
  else if(normalized >= 45) return "D+";
  else if(normalized >= 40) return "D";
  else return "F";
}



// Report Module
void printReport(string name, double relativeGPA, double subjectMarks[], int
subjectCount, double subjectTopperMarks[]) {
```

```
    cout << "\nStudent: " << name << endl;
    cout << "Relative GPA = " << relativeGPA << endl;
    cout << "--Subject Grades—\n";

    for(int i=0;i<subjectCount; i++) {

   cout<<"Subject" << i+1 << " → Marks = " << subjectMarks[i]
        << " → Grade = " << assignGrade(subjectMarks[i], subjectTopperMarks[i])
<< endl;
    }
}

// MAIN FUNCTION
int main() {
    int students;
    cout << "Enter number of students: ";
    cin >> students;

    string names[100];
    double subjectMarks[100][50];
    double subjectCredits[100][50];
    int subjectCount[100];
    double subjectTopperMarks[50] = {0};
    double studentAvg[100];
    double topperAvg = 0;

// Input + Weighted Marks + Average
    for(int s=0; s<students; s++) {

      cout << "\nEnter name of Student " << s+1 << ": ";
      cin >> names[s];

      cout<<"Enter number of subjects for "<<names[s]<<":";
```

```
 cin >> subjectCount[s];

for(int i=0; i<subjectCount[s]; i++) {

  double sessional, mid, final, credit;

  cout << "\nSubject " << i+1 << " credit hours: ";
    cin >> credit;

  cout << "Sessional marks: "; cin >> sessional;
   cout << "Mid marks: "; cin >> mid;
      cout << "Final marks: "; cin >> final;

     subjectMarks[s][i]= calculateMarks(sessional, mid, final);
      subjectCredits[s][i] = credit;

   if(subjectMarks[s][i] > subjectTopperMarks[i]) {
         subjectTopperMarks[i] = subjectMarks[s][i];
      }   }
studentAvg[s] = calculateAverage(subjectMarks[s], subjectCredits[s],
subjectCount[s]);

  if(studentAvg[s] > topperAvg) topperAvg = studentAvg[s];
  }

 // Final Report
 cout << "\n=== Final Report ===\n";

  for(int s=0; s<students; s++) {

   double relativeGPA = calculateRelativeGPA(studentAvg[s], topperAvg);
```

```
printReport(names[s],relativeGPA,subjectMarks[s],subjectCount[s],
subjectTopperMarks);

  }

  return 0;
}
```

❖ Conclusion

- The program successfully demonstrates modular design in C++.
- Each module performs a distinct task: input, calculation, GPA scaling, grade assignment, and reporting.
- The use of arrays and loops allows handling multiple students and subjects efficiently.
- The final report provides a clear transcript with GPA and grades relative to the topper.
- This modular approach improves readability, maintainability, and reusability of code.